

# Implement Client Script & UI Policy (Incident)

## Phase 3 - Project Design Phase Phase

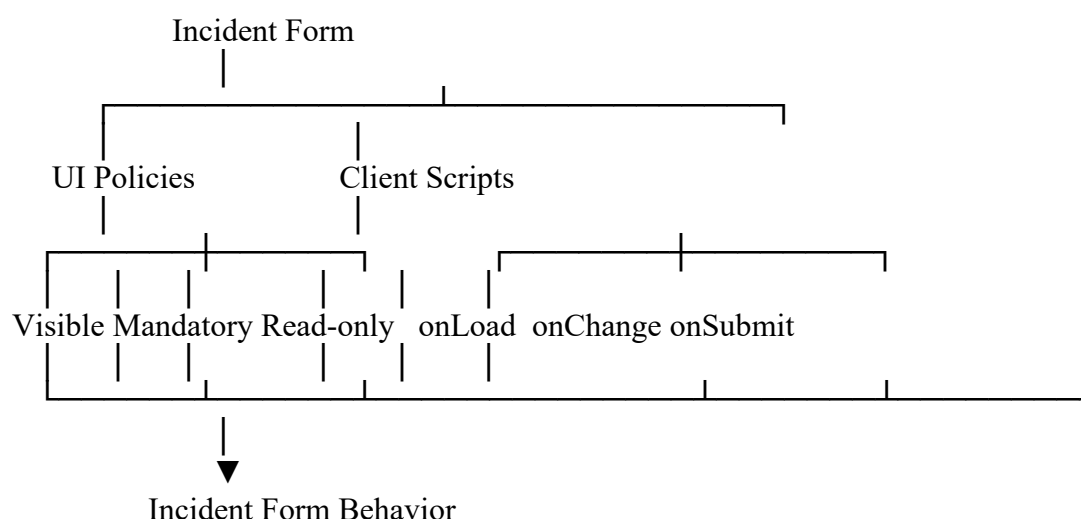
### Project Design Phase — Incident

The **Project Design Phase** converts the analyzed requirements into a concrete ServiceNow implementation design. The goal is to decide **what will be configured, where, and how the components interact** before development begins.

#### 1. Solution Architecture

The Incident form will use a combination of:

- **UI Policies** for straightforward field visibility, mandatory, and read-only behavior.
- **Client Scripts** for dynamic or conditional logic that cannot be handled cleanly by UI Policies.
- **Server-side validation** where a rule must be enforced regardless of the user interface.



#### 2. Proposed Components

| Component                      | Type                   | Purpose                               |
|--------------------------------|------------------------|---------------------------------------|
| Incident Resolution Policy     | UI Policy              | Control resolution-related fields     |
| Incident Classification Policy | UI Policy              | Control Category/Subcategory behavior |
| Incident Initialization        | onLoad Client Script   | Initialize dynamic form behavior      |
| Incident Field Change          | onChange Client Script | React to user changes                 |
| Incident Submission Validation | onSubmit Client Script | Perform final client-side validation  |

The actual components should be reduced if the requirements can be satisfied without them. **Avoid creating a Client Script simply because JavaScript is available.**

### 3. UI Policy Design

#### UI Policy: Resolution Information

**Table:** Incident

**Condition:**

State = Resolved

**Actions:**

Resolution code

Mandatory = True

Resolution notes

Mandatory = True

**Design decision:**

- Use a UI Policy rather than JavaScript.
  - Enable **Reverse if false** if the fields should return to their normal state when the Incident leaves the Resolved state.
- 

### 4. Category/Subcategory Design

If the requirement is:

Subcategory should only be available after Category has been selected.

The design can be:

Category selected



Subcategory becomes available



User selects appropriate Subcategory

If this is purely visibility/mandatory behavior, use a **UI Policy**.

If the requirement involves dynamically determining available values, use an **onChange Client Script** or an appropriate reference/filtering mechanism.

---

### 5. Client Script Design

## onLoad Client Script

### Purpose:

Initialize behavior when an Incident is opened.

Potential responsibilities:

- Evaluate the current Incident state.
- Initialize dynamic field behavior.
- Handle existing Incidents correctly.

**Important:** Don't duplicate UI Policy behavior in the script unless there is a specific reason.

---

## onChange Client Script

**Field:** Category

### Purpose:

React immediately when the user changes the Category.

Conceptually:

Category changes  
↓  
Determine selected Category  
↓  
Apply appropriate behavior  
↓  
Update dependent fields

For example, if changing Category requires clearing an incompatible Subcategory:

Category changed  
↓  
Is current Subcategory still valid?  
↓  
No  
↓  
Clear Subcategory

This prevents stale or inconsistent form data.

---

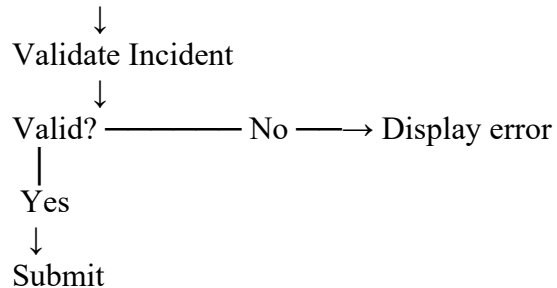
## onSubmit Client Script

### Purpose:

Perform client-side validation immediately before submission.

Conceptually:

User clicks Save/Update



The script should return false when the submission must be stopped.

## 6. Responsibility Matrix

| Behavior                            | UI Policy | Client Script | Server-side |
|-------------------------------------|-----------|---------------|-------------|
| Show/hide field                     | ✓         | Sometimes     | ✗           |
| Make field mandatory                | ✓         | Sometimes     | ✗           |
| Make field read-only                | ✓         | Sometimes     | ✗           |
| Simple state-based UI behavior      | ✓         | ✗             | ✗           |
| Complex dynamic behavior            | ✗         | ✓             | ✗           |
| Client-side submission validation   | ✗         | ✓             | ✗           |
| Security/data integrity enforcement | ✗         | ✗             | ✓           |

## 7. Naming Convention

Use consistent names so administrators can understand the configuration later.

For example:

UI Policy:

INC - Resolved - Resolution Fields Mandatory

Client Script:

INC - Category - Update Dependent Fields

Client Script:

INC - Submit - Validate Resolution

The exact naming convention can be adapted to the organization's existing standards.

## 8. Design Principles

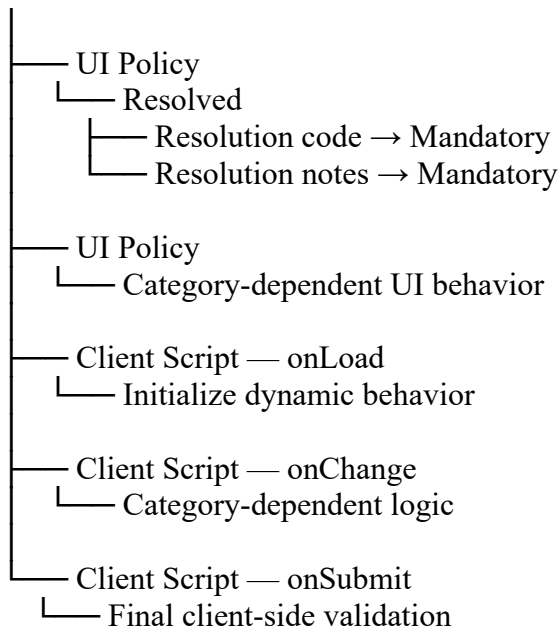
The project should follow these principles:

1. **Configuration before customization** — use UI Policies where possible.
2. **Single responsibility** — each Client Script should perform one logical task.
3. **No unnecessary duplication** — don't implement the same condition in multiple places.
4. **Clear naming** — names should describe the business behavior.
5. **Reversible behavior** — account for users changing values back.
6. **Existing-record support** — behavior should work when opening an existing Incident, not just creating one.
7. **Server-side enforcement where necessary** — client-side logic should not be treated as a security mechanism.
8. **Test dependencies** — particularly Category → Subcategory and State → Resolution fields.

## 9. Project Design Deliverable

At the end of the design phase, the implementation should have a specification similar to:

Incident



This design becomes the blueprint for the **Development/Implementation Phase**, where the actual UI Policies, Client Scripts, conditions, scripts, and test cases are created.